

# pdf loader

RS · 23th September 2026



## Contenido

PyPDF Loader

PDF Miner

PDF Plumber

In [1]:

```
from langchain_community.document_loaders.pdf import PyPDFLoader
from pathlib import Path
from pprint import pp
```

Out [1]:

C:\Users\Uttam\AppData\Local\Temp\ipykernel\_1584\3483426469.py:1: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See <https://github.com/langchain-ai/langchain-community/issues/674> for details and migration guidance toward standalone integration packages.

```
from langchain_community.document_loaders.pdf import PyPDFLoader
```

In [2]:

```
# create the path for pdf file
```

```
file_path = Path("../knowledge-source/attention_is_all_you_need.pdf")
```

```
file_path.exists()
```

Out [2]:

```
True
```

## PyPDF Loader

In [3]:

```
file_path.as_posix()
```

Out [3]:

```
'../knowledge-source/attention_is_all_you_need.pdf'
```

In [4]:

```
# create a loader
```

```
pypdf_loader = PyPDFLoader(file_path=file_path.as_posix(),
                             mode='page')
```

In [5]:

```
# Load the documents
```

```
documents = pypdf_loader.load()
```

In [6]:

```
# number of documents
```

```
len(documents)
```

Out [6]:

```
15
```

In [14]:

```
pp(documents[2].metadata)
```

Out [14]:

```
{'producer': 'pdfTeX-1.40.25',
 'creator': 'LaTeX with hyperref',
 'creationdate': '2024-04-10T21:11:43+00:00',
 'author': '',
 'keywords': '',
 'moddate': '2024-04-10T21:11:43+00:00',
 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live '
                    '2023) kpathsea version 6.3.5',
 'subject': '',
 'title': '',
 'trapped': '/False',
 'source': '../knowledge-source/attention_is_all_you_need.pdf',
 'total_pages': 15,
 'page': 2,
 'page_label': '3'}
```

In [13]:

```
# fetch the first page text content
```

```
print(documents[2].page_content)
```

Out [13]:

Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

### 3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of  $N = 6$  identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is  $\text{LayerNorm}(x + \text{Sublayer}(x))$ , where  $\text{Sublayer}(x)$  is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension  $d_{\text{model}} = 512$ .

Decoder: The decoder is also composed of a stack of  $N = 6$  identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position  $i$  can depend only on the known outputs at positions less than  $i$ .

### 3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

In [9]:

```
# metadata
```

```
pp(documents[4].metadata)
```

Out [9]:

```
{'producer': 'pdfTeX-1.40.25',
 'creator': 'LaTeX with hyperref',
 'creationdate': '2024-04-10T21:11:43+00:00',
 'author': '',
 'keywords': '',
 'moddate': '2024-04-10T21:11:43+00:00',
 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live '
                    '2023) kpathsea version 6.3.5',
 'subject': '',
 'title': '',
 'trapped': '/False',
 'source': '../knowledge-source/attention_is_all_you_need.pdf',
 'total_pages': 15,
 'page': 4,
 'page_label': '5'}
```

In [15]:

```
from langchain_community.document_loaders.parsers import TesseractBlobParser, RapidOCRBlobParser
```

In [16]:

```
# create pypdf instance which can extract images
```

```
pypdf_image_loader = PyPDFLoader(file_path=file_path.as_posix(),
                                  mode="page",
                                  extract_images=True,
                                  images_parser=RapidOCRBlobParser(),
                                  images_inner_format="html-img")
```

In [17]:

```
# Load documents with images
```

```
documents_with_images = pypdf_image_loader.load()
```

Out [17]:

```
[32m[INFO] 2026-09-23 21:22:31,725 [RapidOCR] base.py:23: Using engine_name: onnxruntime[0m
[32m[INFO] 2026-09-23 21:22:32,067 [RapidOCR] download_file.py:60: File exists and is valid: E:\Projects\Campusx-Advance-
Rag\Code\.venv\Lib\site-packages\rapidocr\models\PP-OCrv6_det_small.onnx[0m
[32m[INFO] 2026-09-23 21:22:32,068 [RapidOCR] main.py:63: Using E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-
packages\rapidocr\models\PP-OCrv6_det_small.onnx[0m
[32m[INFO] 2026-09-23 21:22:32,286 [RapidOCR] base.py:23: Using engine_name: onnxruntime[0m
[32m[INFO] 2026-09-23 21:22:32,309 [RapidOCR] download_file.py:60: File exists and is valid: E:\Projects\Campusx-Advance-
Rag\Code\.venv\Lib\site-packages\rapidocr\models\ch_ppocr_mobile_v2.0_cls_mobile.onnx[0m
[32m[INFO] 2026-09-23 21:22:32,309 [RapidOCR] main.py:63: Using E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-
packages\rapidocr\models\ch_ppocr_mobile_v2.0_cls_mobile.onnx[0m
[32m[INFO] 2026-09-23 21:22:32,427 [RapidOCR] base.py:23: Using engine_name: onnxruntime[0m
[32m[INFO] 2026-09-23 21:22:32,517 [RapidOCR] download_file.py:60: File exists and is valid: E:\Projects\Campusx-Advance-
Rag\Code\.venv\Lib\site-packages\rapidocr\models\PP-OCrv6_rec_small.onnx[0m
[32m[INFO] 2026-09-23 21:22:32,517 [RapidOCR] main.py:63: Using E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-
packages\rapidocr\models\PP-OCrv6_rec_small.onnx[0m
```

In [18]:

```
# Length of the document
```

```
len(documents_with_images)
```

Out [18]:

```
15
```

In [19]:

```
page_with_image = documents_with_images[2]
```

In [20]:

```
print(page_with_image.page_content[-700:])
```

Out [20]:

embeddings are offset by one position, ensures that the  
 predictions for position  $i$  can depend only on the known outputs at positions less than  $i$ .

### 3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

3

```
<img alt="Output
Probabilities
Softmax
Linear
Add & Norm
Feed
Forward
Add & Norm
Add & Norm
Multi-Head
Feed
Attention
Forward" data-bbox="59 793 169 940"/>
```

```

Nx
Add & Norm
Nx
Add & Norm
Masked
Multi-Head
Multi-Head
Attention
Attention
Positional
Positional
Encoding
Encoding
Input
Output
Embedding
Embedding
Inputs
Outputs
(shifted right) src="#" />

```

## PDF Miner

```

In [21]:
from langchain_community.document_loaders import PDFMinerLoader

```

```

In [22]:
# create the miner loader

pdfminer_loader = PDFMinerLoader(file_path=file_path.as_posix(),
                                  mode="page",
                                  extract_images=True,
                                  images_parser=RapidOCRBlobParser(),
                                  images_inner_format="html-img")

```

```

In [23]:
# Load the documents

documents_with_images = pdfminer_loader.load()

```

```

Out [23]:
[32m[INFO] 2026-09-23 21:24:41,535 [RapidOCR] base.py:23: Using engine_name: onnxruntime[0m
[32m[INFO] 2026-09-23 21:24:41,572 [RapidOCR] download_file.py:60: File exists and is valid: E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-packages\rapidocr\models\PP-OCrv6_det_small.onnx[0m
[32m[INFO] 2026-09-23 21:24:41,573 [RapidOCR] main.py:63: Using E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-packages\rapidocr\models\PP-OCrv6_det_small.onnx[0m
[32m[INFO] 2026-09-23 21:24:41,725 [RapidOCR] base.py:23: Using engine_name: onnxruntime[0m
[32m[INFO] 2026-09-23 21:24:41,729 [RapidOCR] download_file.py:60: File exists and is valid: E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-packages\rapidocr\models\ch_ppocr_mobile_v2.0_cls_mobile.onnx[0m
[32m[INFO] 2026-09-23 21:24:41,730 [RapidOCR] main.py:63: Using E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-packages\rapidocr\models\ch_ppocr_mobile_v2.0_cls_mobile.onnx[0m
[32m[INFO] 2026-09-23 21:24:41,860 [RapidOCR] base.py:23: Using engine_name: onnxruntime[0m
[32m[INFO] 2026-09-23 21:24:41,934 [RapidOCR] download_file.py:60: File exists and is valid: E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-packages\rapidocr\models\PP-OCrv6_rec_small.onnx[0m
[32m[INFO] 2026-09-23 21:24:41,935 [RapidOCR] main.py:63: Using E:\Projects\Campusx-Advance-Rag\Code\.venv\Lib\site-packages\rapidocr\models\PP-OCrv6_rec_small.onnx[0m

```

```

In [24]:
# image text in pdf

print(documents_with_images[2].page_content[-450:])

```

```

Out [24]:
n output,
where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

3



```

In [25]:

```
# text from table
```

```
print(documents_with_images[5].page_content)
```

Out [25]:

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types.  $n$  is the sequence length,  $d$  is the representation dimension,  $k$  is the kernel size of convolutions and  $r$  the size of the neighborhood in restricted self-attention.

Layer Type

Complexity per Layer

Self-Attention

Recurrent

Convolutional

Self-Attention (restricted)

$O(n^2 \cdot d)$

$O(n \cdot d^2)$

$O(k \cdot n \cdot d^2)$

$O(r \cdot n \cdot d)$

Sequential Maximum Path Length

Operations

$O(1)$

$O(n)$

$O(1)$

$O(1)$

$O(1)$

$O(n)$

$O(\log k(n))$

$O(n/r)$

### 3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension  $d_{model}$  as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

In this work, we use sine and cosine functions of different frequencies:

$$P E(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$P E(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

where  $pos$  is the position and  $i$  is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from  $2\pi$  to  $10000 \cdot 2\pi$ . We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset  $k$ ,  $P E(pos+k)$  can be represented as a linear function of  $P E(pos)$ .

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

#### 4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations  $(x_1, \dots, x_n)$  to another sequence of equal length  $(z_1, \dots, z_n)$ , with  $x_i, z_i \in \mathbb{R}^d$ , such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [12]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires  $O(n)$  sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence

6

In [26]:

```
pp(documents_with_images[0].metadata)
```

Out [26]:

```
{'producer': 'pdfTeX-1.40.25',
 'creator': 'LaTeX with hyperref',
 'creationdate': '2024-04-10T21:11:43+00:00',
 'author': '',
 'keywords': '',
 'moddate': '2024-04-10T21:11:43+00:00',
 'ptex.fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live '
                    '2023) kpathsea version 6.3.5',
 'subject': '',
 'title': '',
 'trapped': 'False',
 'total_pages': 15,
 'source': '../knowledge-source/attention_is_all_you_need.pdf',
 'page': 0}
```

In [27]:

```
for doc in documents_with_images:
    if doc.metadata["page"] == 7:
        print(doc.page_content)
        break
```

Out [27]:

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model

ByteNet [18]

Deep-Att + PosUnk [39]

GNMT + RL [38]

ConvS2S [9]  
 MoE [32]  
 Deep-Att + PosUnk Ensemble [39]  
 GNMT + RL Ensemble [38]  
 ConvS2S Ensemble [9]  
 Transformer (base model)  
 Transformer (big)

BLEU

Training Cost (FLOPs)

EN-DE EN-FR  
 23.75

24.6  
 25.16  
 26.03

26.30  
 26.36  
 27.3  
 28.4

39.2  
 39.92  
 40.46  
 40.56  
 40.4  
 41.16  
 41.29  
 38.1  
 41.8

EN-DE

EN-FR

$2.3 \cdot 10^{19}$   
 $9.6 \cdot 10^{18}$   
 $2.0 \cdot 10^{19}$

$1.8 \cdot 10^{20}$   
 $7.7 \cdot 10^{19}$

$1.0 \cdot 10^{20}$   
 $1.4 \cdot 10^{20}$   
 $1.5 \cdot 10^{20}$   
 $1.2 \cdot 10^{20}$   
 $8.0 \cdot 10^{20}$   
 $1.1 \cdot 10^{21}$   
 $1.2 \cdot 10^{21}$

$3.3 \cdot 10^{18}$   
 $2.3 \cdot 10^{19}$

**Residual Dropout** We apply dropout [33] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of  $P_{drop} = 0.1$ .

**Label Smoothing** During training, we employed label smoothing of value  $\epsilon_{ls} = 0.1$  [36]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

## 6 Results

### 6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used

dropout rate  $P_{\text{drop}} = 0.1$ , instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty  $\alpha = 0.6$  [38]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [38].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU 5.

## 6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the

5We used values of 2.8, 3.7, 6.0 and 9.5 TFLOPS for K80, K40, M40 and P100, respectively.

8

## PDF Plumber

In [28]:

```
from langchain_community.document_loaders import PDFPlumberLoader
```

In [29]:

```
# create the loader
```

```
plumber_loader = PDFPlumberLoader(file_path=file_path.as_posix())
```

In [30]:

```
# Load the docs
```

```
documents_with_metadata = plumber_loader.load()
```

In [31]:

```
len(documents_with_metadata)
```

Out [31]:

15

In [32]:

```
print(documents_with_metadata[1].page_content)
```

Out [32]:

### 1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state-of-the-art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the position to steps in computation time, they generate a sequence of hidden states  $h_t$  as a function of the previous hidden state  $h_{t-1}$  and the input for position  $t$ . This inherently

sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state-of-the-art in

translation quality after being trained for as little as twelve hours on eight P100 GPUs.

## 2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU

[16], ByteNet [18] and ConvS2S [9], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [12]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

Self-attention, sometimes called intra-attention, is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 27, 28, 22]. End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple language question answering and language modeling tasks [34].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over modelssuch as [17, 18] and [9].

## 3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35].

Here, the encoder maps an input sequence of symbol representations  $(x_1, \dots, x_n)$  to a sequence

of continuous representations  $z = (z_1, \dots, z_n)$ . Given  $z$ , the decoder then generates an output

sequence  $(y_1, \dots, y_m)$  of symbols one element at a time. At each step the model is auto-regressive

[10], consuming the previously generated symbols as additional input when generating the next.

2

In [33]:

```
pp(documents_with_metadata[0].metadata)
```

Out [33]:

```
{'source': '../knowledge-source/attention_is_all_you_need.pdf',
 'file_path': '../knowledge-source/attention_is_all_you_need.pdf',
 'page': 0,
 'total_pages': 15,
 'Author': '',
 'CreationDate': 'D:20240410211143Z',
 'Creator': 'LaTeX with hyperref',
 'Keywords': '',
 'ModDate': 'D:20240410211143Z',
 'PTEX.Fullbanner': 'This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live '
                    '2023) kpathsea version 6.3.5',
 'Producer': 'pdfTeX-1.40.25',
 'Subject': '',
 'Title': '',
 'Trapped': 'False'}
```